

1. Same Tree

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:

Input: p = [1,2,3], q = [1,2,3]

Output: true

Example 2:

Input: p = [1,2], q = [1,null,2]

Output: false

Example 3:

Input: p = [1,2,1], q = [1,1,2]

Output: false

Constraints:

The number of nodes in both trees is in the range [0, 100].

$-104 \leq \text{Node.val} \leq 104$

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int val) { this.val = val; }
}

public class SameTree {
    // Main logic function
    public boolean isSameTree(TreeNode p, TreeNode q) {
        // If both nodes are null, trees are identical at this point
        if (p == null && q == null) return true;

        // If one is null or values differ, trees are not identical
        if (p == null || q == null || p.val != q.val) return false;

        // Recursively check left and right subtrees
        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

```

    }

    // Main method for testing
    public static void main(String[] args) {
        SameTree st = new SameTree();

        // Example 1: p = [1,2,3], q = [1,2,3]
        TreeNode p1 = new TreeNode(1);
        p1.left = new TreeNode(2);
        p1.right = new TreeNode(3);

        TreeNode q1 = new TreeNode(1);
        q1.left = new TreeNode(2);
        q1.right = new TreeNode(3);

        System.out.println("Example 1: " + st.isSameTree(p1, q1)); // Output: true
    }
}

```

2.Symmetric Tree

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Example 1:

Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:

Input: root = [1,2,2,null,3,null,3]

Output: false

Constraints:

The number of nodes in the tree is in the range [1, 1000].

$-100 \leq \text{Node.val} \leq 100$

```
import java.util.*;
```

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
}
```

```

    TreeNode(int val) {
        this.val = val;
    }
}

public class SymmetricTree {
    public boolean isSymmetric(TreeNode root) {
        if (root == null) return true;
        return checkMirror(root.left, root.right);
    }

    private boolean checkMirror(TreeNode left, TreeNode right) {
        if (left == null && right == null) return true;
        if (left == null || right == null || left.val != right.val) return false;

        // Mirror check: Outer children together, Inner children together
        return checkMirror(left.left, right.right) && checkMirror(left.right,
right.left);
    }

    public static void main(String[] args) {
        SymmetricTree sol = new SymmetricTree();

        // Example 1: [1,2,2,3,4,4,3] -> Symmetric
        TreeNode root1 = new TreeNode(1);
        root1.left = new TreeNode(2);
        root1.right = new TreeNode(2);
        root1.left.left = new TreeNode(3);
        root1.left.right = new TreeNode(4);
        root1.right.left = new TreeNode(4);
        root1.right.right = new TreeNode(3);

        // Example 2: [1,2,2,null,3,null,3] -> NOT Symmetric
        TreeNode root2 = new TreeNode(1);
        root2.left = new TreeNode(2);
        root2.right = new TreeNode(2);
        root2.left.right = new TreeNode(3); // This is an "inner" node
        root2.right.right = new TreeNode(3); // This is an "outer" node

        // Explicitly printing both
        System.out.println(sol.isSymmetric(root1));
        System.out.println(sol.isSymmetric(root2));
    }
}

```

3.

Medium

Topics

premium lock icon

Companies

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

Input: grid =

```
[[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],  
[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,0,1,0],  
[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0]]
```

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0,0]]

Output: 0

Constraints:

$m == \text{grid.length}$

$n == \text{grid}[i].\text{length}$

$1 \leq m, n \leq 50$

$\text{grid}[i][j]$ is either 0 or 1.